



Roskilde University

Portfolio Subproject 3

COURSE

COMPLEX IT SYSTEMS

Group 11 members:

IDA HAY JØRGENSEN	80088
JULIUS KRÜGER MADSEN	80734
MAREK LASLO	82428
SOFUS HILFLING NIELSEN	82278

Source Code Repository:

https://github.com/RUC-MSc-CS-CIT-2024/portfolio_subproject_3

December 18, 2024

Contents

1	Introduction	3
2	Application design	3
2.1	Wireframing	3
2.2	Routes overview	5
2.2.1	Path "/"	6
2.2.2	Path "/search"	6
2.2.3	Path "/signup"	6
2.2.4	Path "/profile"	6
2.2.5	Path "/profile/lists"	7
2.2.6	Path "/media/:id"	7
2.2.7	Path "/persons/:id"	7
2.2.8	Path "/*"	7
3	Development	7
3.1	React Components	7
3.1.1	Layout	8
3.1.2	Navigation bar	8
3.1.3	Media and Person Card	9
3.1.4	Media and Person Carousels	9
3.1.5	Search Form	9
3.1.6	Toast notification	9
3.2	React patterns	9
3.2.1	Hooks	9
3.2.2	DOM Router	10
3.2.3	Preventing prop drilling	11
3.3	Component hierarchy	11
3.4	React-Bootstrap	11
3.5	Internal Data Representation and CRUD Operations	12
3.5.1	Data Representation as Objects	13
3.5.2	CRUD Service Functions	13
3.5.3	Integration with The Movie Database (TMDB)	14

3.6	Utilities and helper functions	15
3.6.1	cookie.js	15
3.6.2	crew.js	16
3.6.3	date.js	16
3.6.4	debounce.js	17
3.6.5	fetchAllPages.js	17
3.6.6	getUserFromSession.js	18
3.6.7	validation.js	19
4	Reflection	19
5	Appendix A	20
5.1	Ida Hay Jørgensen	20
5.2	Julius Krüger Madsen	26
5.2.1	Database	26
5.2.2	Backend	27
5.2.3	Frontend	28
5.3	Marek Laslo	29
5.4	Sofus Hilfling Nielsen	31
6	Appendix B	33
6.1	Wireframes	33
6.2	ApiClient code	41
6.3	Changes to the database and the backend	45
6.3.1	Portfolio subproject 1 - Database	45
6.3.2	Portfolio subproject 2 - Backend	46

1 Introduction

This report documents the design, implementation and evaluation of the front-end application developed for the 3rd section in the course Complex IT Systems. This sub-project builds upon the backend functionality implemented in Portfolio Subproject 2 and the database functionality created in the Portfolio Subproject 1. There has been adjusted some functionality in the to prior section and this is documented in the appendix 6.3.1 and 6.3.2. Along with these changes the user provided in the prior sections has a new password. Login credentials username: john_doe, password: password123456789.

The focus in this project is on the creation of a graphical user interface that allows users to interact with the system. The application is designed to be a modular and scalable single-page application (SPA) using modern technologies, including React, JavaScript, and Bootstrap. The front-end application is structured with a multi-layered architecture, ensuring a clear separation of concerns.

2 Application design

2.1 Wireframing

The application design began with creating wireframes to ensure a user-centred design. Using Jakob Nielsen's 10 usability heuristics provided a framework for ensuring an intuitive and effective design (Nielsen, 1994). The developed low-fidelity wireframes are available for reference here (Own-work, 2024) and in appendix 6.1.

The application features a consistent navigation bar that maintains a uniform appearance across all pages, while dynamically changing content within the main section. See figure 1. This design follows Nielsen's 4th and 8th heuristics of keeping a consistent and minimalist design. The navigation bar includes a link to the home page via the application's name, helping users recover from errors and navigate easily. This approach aligns with Nielsen's 3rd, 5th and 7th heuristics, ensuring user control and freedom to correct mistakes.

Another critical aspect of the design is providing clear feedback for user actions, whether successful or wrong. Toast notifications are used to deliver this feedback, using natural language for clarity, in line with Nielsen's 2nd and 9th heuristics.

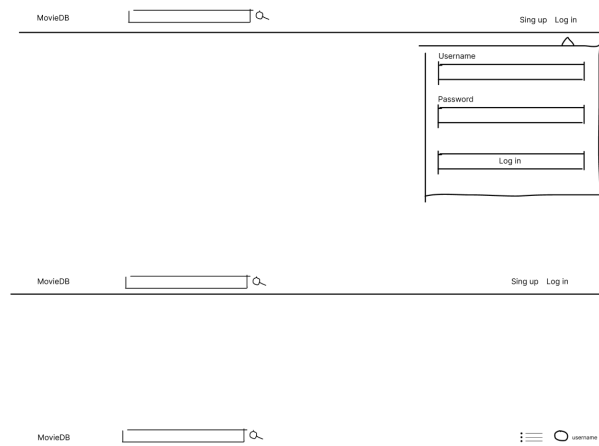


Figure 1: *Low fi navigation bar design, (Own-work, 2024).*

For example, login and sign-up actions have concise messages that confirm the outcome of the action. These features align with Nielsen's 1st and 6th heuristics, which emphasize visibility of system status and error prevention. Examples of these components can be seen in figure 2 and figure 3.

Log in

Username

Password

Log in

Sign up

Email

Username

Password

Sign up

Figure 2: Low fi log in and sign up design, (Own-work, 2024).

MovieDB

☰ user

Profile settings

Username

Email

Password

Update profile

Delete user
Log out

Success: you changed your data

Figure 3: Low fi profile page design, (Own-work, 2024).

2.2 Routes overview

This section outlines the routes defined within the application and includes a site structure diagram in figure 4, providing an overview of the routes.

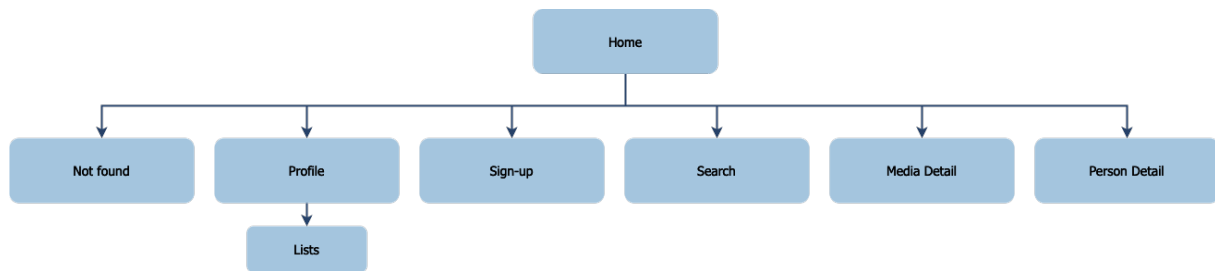


Figure 4: Site structure diagram.

This diagram is crucial for visualizing and organizing the application's structure, serving as a foundational element in designing an intuitive site that enables users to navigate easily (Anderson, 2023). The routes are explained in detail in the following.

2.2.1 Path "/"

When visiting the movie portfolio website, this is the first path the user is directed to. Here, the user is welcomed with a search form.

2.2.2 Path "/search"

After the user types and search query for a media or person, they are redirected to the search page. This path is dynamically extended with a query parameter reflecting the search term. For example, the final URL may appear as `"/search?query=SearchWord"`.

2.2.3 Path "/signup"

If users wish to access the full functionality of the website, they are encouraged to create an account. Selecting this option redirects them to the sign-up page, located at the `"/signup"` path, where they can register for a personalized experience.

2.2.4 Path "/profile"

A new user is created, and with this, they gain access to the `"/profile"` path. This route is exclusive to logged-in users, providing them with the ability to view and edit their profile information.

2.2.5 Path `"/profile/lists"`

The path for user profiles has a sub-path that directs users to their lists. These lists include the following, bookmarked and completed media. Users can view, organize, and manage their saved content, including items they have rated or wish to watch in the future. To access the specific tab for following, bookmarked or completed an anchor can be used after the path e.g. `"/profile/lists#following"`.

2.2.6 Path `"/media/:id"`

This path allows users to access detailed information about a specific media item by specifying its unique ID. This route is triggered when a user selects a media card or enters a valid media ID. On this page, users can view detailed information, bookmark, complete and rate the media, or explore related content.

2.2.7 Path `"/persons/:id"`

This route is accessed when a user selects a person from the search results or a person's card elsewhere in the application. It provides information about the person and the ability to follow the person if logged in.

2.2.8 Path `"/*"`

Any other undefined paths are treated as invalid and will redirect users to the 404 "Not Found" page, ensuring a consistent experience when encountering non-existent routes.

3 Development

3.1 React Components

To keep the movie database frontend modular and to apply the principle of separation of concerns within this project, the use of React components was encouraged during

the development of the portfolio project. This approach not only promotes cleaner and more maintainable code but simplifies debugging and testing by isolating specific functionalities within individual components.

The entire project is built from small, reusable functional components, which are then combined into larger components and ultimately displayed as a page layout. Each component is designed to handle a specific responsibility, such as rendering a media card, displaying a grid of movies, or managing navigation elements. By breaking down the UI into these smaller parts, the project ensures scalability and easier updates in the future.

Additionally, React's declarative nature and component life-cycle management allow for seamless integration of state and props, making it possible to create a dynamic and responsive user interface. This philosophy improves development efficiency and ensures a consistent user experience across all pages of the application.

The following highlights some of the components developed for the application, showing different implementations of React components.

3.1.1 Layout

Layout a component, which is connecting the content of a page with a navigation bar on top. It has its place on top of a DOM tree, and all the other components are rendered within the layout component. It behaves like an empty shell with a navigation bar, where the contents of individual pages are being rendered within.

3.1.2 Navigation bar

As there is only one navigation bar instance being used throughout the application, the need for creating it in a separate component was not that immense, but to achieve better scalability the component for it was created. It has its place on top of the hierarchy within a Layout component.

3.1.3 Media and Person Card

These two components are used as a minimalist way to display critical details about the media or a person's entity. These components are used in every part of the application, when searching for a title or a person, or when seeing similar media recommendations on a media page.

3.1.4 Media and Person Carousels

On the other hand, these components work as containers with many cards. They group cards and display different media or persons. Another feature of these components is the option of switching to a set of different cards by clicking the left and right arrows on the carousels.

3.1.5 Search Form

The search form is a component that takes a string as input and triggers an event when the search button is pressed. This component is used on different pages and in the navigation bar. Its main usage is that users can type and search titles and persons.

3.1.6 Toast notification

This component is used as an information provider when something goes wrong or when a process is successful. For instance, during a successful or failed login attempt.

3.2 React patterns

How do patterns in react differ from traditional patterns in the backend?

3.2.1 Hooks

Hooks are functions built into React that enable you to interact with the React engine. The most common hook is the `useState` hook that allows you to store a state

between component re-renders. Another commonly used hook is the `useEffect` and `useCallback` hooks, they are both used to interact with things outside of react. The `useEffect` is used to automatically rerun a piece of code if one of its dependencies changes, whereas the callback encapsulates a specific function to do the same. The last common hook type is the `useContext` hook, this hook interacts with a react context which is a component that allows for sharing state between components without passing properties between them. The frontend uses multiple contexts for handling user authentication as well as search queries and toast management.

Custom hooks enable you to move functionality into reusable functions that can be reused in other components, the difference between a hook and a function is that a hook allows you to use built-in react hooks like a component does, this is not allowed in a normal function. Custom functions are used in the frontend to simplify authentication handling. By creating a custom hook, `useAuth`, the logic for interacting with the `AuthContext` provider is abstracted, enabling the decoupling of authentication logic from rendering logic. Other custom hooks are `usePagainatedData` and `useItemsPerRow` which simplify pagination and rendering of items in the grid and carousel components respectively.

3.2.2 DOM Router

In Single-page applications (SPAs) HTML content is rewritten instead of reloading the entire page from the server whenever something changes (Developer Mozilla, 2024b). This requires the browser to handle navigation between pages differently compared to traditional sites where the user is navigated back to the previous page from the server, this concept is referred to as DOM routing.

React Router is a versatile component library designed primarily for DOM routing, while also supporting other types of routing. By routing client-side in the browser the user can experience a better experience as the structure of the site already has been downloaded on the initial connection to the site. This allows only refreshing parts of the site that contain dynamic content like the media details without having to refresh static content like the navigation bar or input fields like the search form on the search page.

3.2.3 Preventing prop drilling

If prop drilling occurs, it can result in tightly coupled components, reduced code readability, and increased difficulty in maintaining the code base.

To address this challenge, the application uses various strategies to minimize prop drilling. By utilizing contexts, shared data is centralized, allowing components to access the required data directly without the need to pass it through multiple layers. Additionally, for the components, the data being sent also represents state information. For example, the absence of an ID in the media or person card component signifies that the data is either still loading or there was an error in fetching it.

3.3 Component hierarchy

The following diagram illustrates the component hierarchy, which highlights the structure and relationships between the different elements of the application, see figure 5.

3.4 React-Bootstrap

To optimize resource usage and production time during the design of the movie database, and to eliminate the need for manual styling of each element and component, the CSS framework Bootstrap was chosen as the project's design framework.

To integrate the framework into the application, the `react-bootstrap` library was installed. Once installed, basic components such as forms and buttons could be easily imported into the relevant files from the `react-bootstrap` library.

This library not only provides Bootstrap-styled components but also offers a wide range of pre-styled classes, which can be used to style elements in the application without the need to write custom style sheets.

By using a framework like this, the development process is accelerated, developer resources are freed up, a unified design across the project is maintained, and the creation of a responsive layout for the product becomes significantly easier.

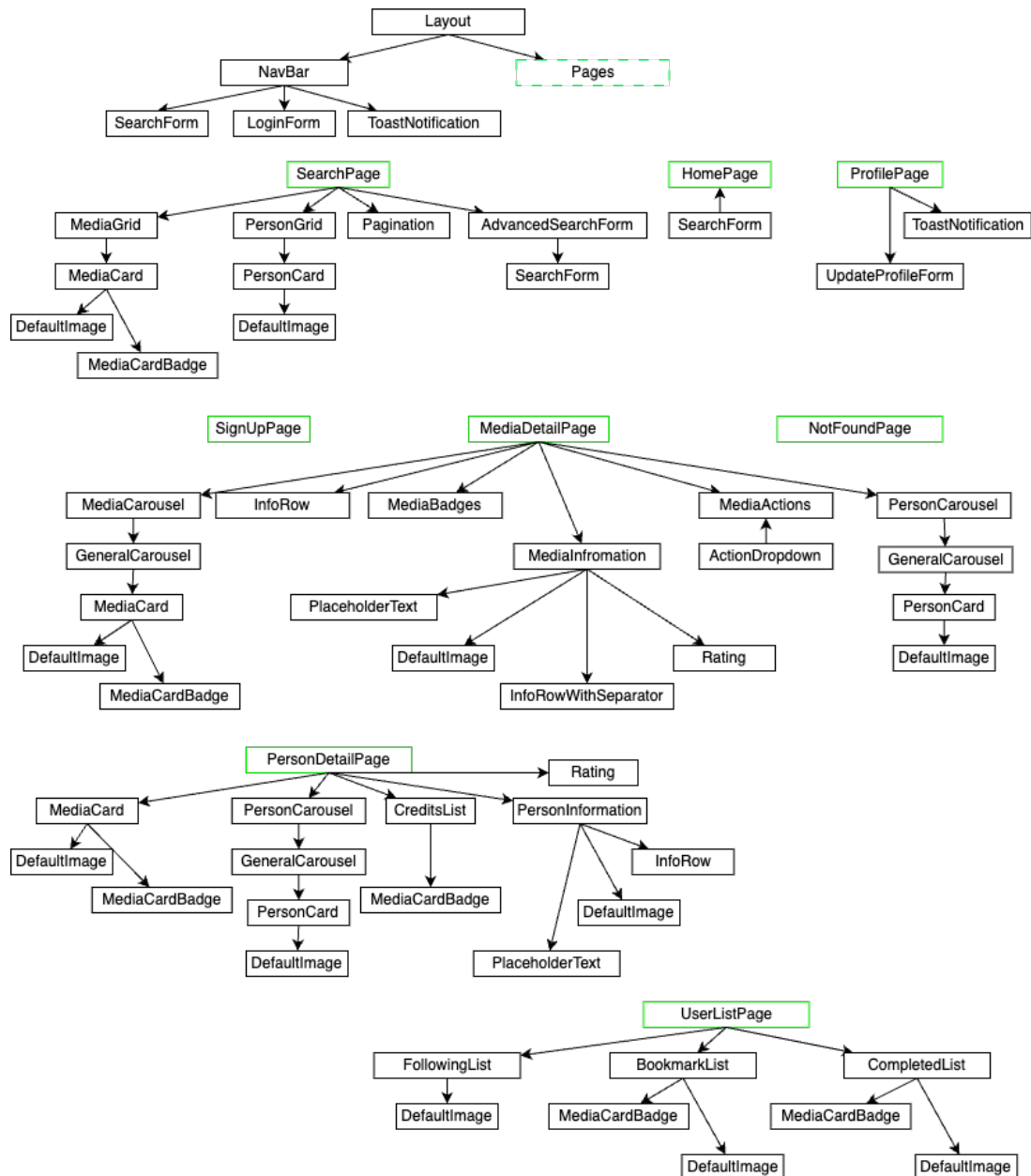


Figure 5: Component hierarchy diagram.

3.5 Internal Data Representation and CRUD Operations

This section discusses the internal representation of data within the application, focusing on the classes and functions used for managing operations such as Create, Read, Update, and Delete (CRUD). Additionally, it explores the integration with external APIs, such as The Movie Database (TMDB).

3.5.1 Data Representation as Objects

The application uses an object-oriented approach for handling data and encapsulating related properties and methods within cohesive units. Data entities such as users, media, and persons are represented as JavaScript objects, with functions managing their state and interactions.

ApiClient: Central to the application's architecture is the `ApiClient`, a reusable utility class designed to interact with the backend API. It simplifies HTTP requests through methods like `Get`, `Post`, `Patch`, and `Delete`. Each method abstracts the complexities of constructing requests, handling responses, and managing errors. An example instantiation is shown in code 1.

```
const api = new ApiClient(baseUrl, accessKey);
const response = await api.Get(`/media/${mediaId}`);
if (!response.ok) {
  throw new Error('Failed to fetch media.');
```

```
}
```

```
const media = response.value;
```

Code 1: ApiClient usage example.

This utility is used vastly across services, such as `mediaService` and `personService`, to ensure consistency and reliability in data transactions.

3.5.2 CRUD Service Functions

Service functions encapsulate the logic for performing CRUD operations on data entities. Examples include:

- **mediaService:** Handles operations related to media entities, including fetching media records with functions such as `fetchMediaCast` and `fetchMediaById`.
- **personService:** Provides methods like `fetchPersons` and `fetchPersonById` to manage person-related data, integrating external TMDB data for enrichment.

- **userService:** Manages user-centric operations, including `createBookmark`, `updateUserById`, and `getCurrentUserBookmarks`.

The modular nature of these services ensures the separation of concerns and maintains clean code practices.

3.5.3 Integration with The Movie Database (TMDB)

The application integrates with TMDB for enriching media and person data. This is achieved through functions in the `tmdbService`, which utilize TMDB's RESTful API.

Example Functions:

- **getTMDBImage:** Fetches images for media and persons based on their IMDB IDs.
- **fetchPersonTMDB:** Retrieves detailed person information, such as known works and popularity metrics.
- **fetchPersonImages:** Collects high-resolution images of persons for display in the application.

Code snippet 2 demonstrates the process of fetching images from TMDB.

```
export async function getTMDBImage(imdb_id, size) {
  const api = new ApiClient(baseUrl, accessKey);
  size = size ?? ImageSize.Normal;
  const resp = await api.Get(`find/${imdb_id}?external_source=imdb_id`);
  if (!resp.ok) {
    throw new Error('Network response was not ok');
  }
  const path = resp.value.movie_results[0]?.poster_path;
  return new URL(size + '/' + path, baseImageUrl).href;
}
```

Code 2: Fetching TMDB images.

3.6 Utilities and helper functions

Throughout the implementation of the application, the need to reuse certain logic across different parts of the system became necessary. To address this, files containing utility and helper functions were created. The `ApiClient`, which has been previously discussed, serves as an example of this approach. Other helper functions will now be explained.

3.6.1 `cookie.js`

This file contains functions to help set, retrieve and delete cookie information from the browser. This has been used to store and retrieve JWT token information from the logged-in user, see code 3.

```
export const setCookie = (name, value, expireInSeconds) => {
  const expires = new Date(Date.now() + expireInSeconds *
    ↪ 1000).toUTCString();
  document.cookie = `${name}=${encodeURIComponent(value)};
    ↪ expires=${expires}; path=/; Secure`;
};

export const getCookie = (name) => {
  return document.cookie.split('; ').reduce((r, v) => {
    const parts = v.split('=');
    return parts[0] === name ? decodeURIComponent(parts[1]) : r;
  }, '');
};

export const deleteCookie = (name) => {
  setCookie(name, '', -1);
};
```

Code 3: JavaScript cookie management functions.

3.6.2 crew.js

This file contains a function designed to extract crew members from the response. The function filters and retrieves the names of crew members based on specific job categories and roles, providing flexibility through case-insensitive matching and optional role filtering, see code 4.

```
export const extractMembersByJobCategory = (crew, jobCategory, role =  
  null) => {  
  if (!Array.isArray(crew)) {  
    return [];  
  }  
  
  let filteredMembers = crew.filter((member) =>  
    member?.jobCategory?.toLowerCase()  
      .includes(jobCategory.toLowerCase())  
  );  
  
  if (role) {  
    filteredMembers = filteredMembers.filter(  
      (member) => member?.role?.toLowerCase() === role.toLowerCase(),  
    );  
  }  
  
  return filteredMembers.map((member) => member?.personName);  
};
```

Code 4: Extract members by job category.

3.6.3 date.js

The formatDate function takes a date string, converts it to a date object, and then formats it into a readable string using the specified options. see code 5.

```
export const formatDate = (dateString) => {
  const options = { year: 'numeric', month: 'long', day: 'numeric' };
  const date = new Date(dateString);
  return date.toLocaleDateString(
    undefined,
    options
  );
};
```

Code 5: Format date function.

3.6.4 debounce.js

The debounce helper ensures that certain functions only execute after a specified amount of time has passed, preventing multiple executions within that time frame, see code 6.

```
export default function debounce(func, wait) {
  let timeout;
  return (...args) => {
    clearTimeout(timeout);
    timeout = setTimeout(
      () => func.apply(this, args),
      wait
    );
  };
}
```

Code 6: Debounce function.

3.6.5 fetchAllPages.js

The fetchAllPages function is designed to fetch data from multiple pages of an API until there is no more data to fetch. It aggregates all the data into a single array and returns it, see code 7.

```
export const fetchAllPages = async (
  fetchFunction,
  count = 12,
  personId = null,
) => {
  let allData = [];
  let currentPage = 1;
  let hasMoreData = true;

  while (hasMoreData) {
    const result = personId
      ? await fetchFunction(personId, currentPage, count)
      : await fetchFunction(currentPage, count);
    allData = [...allData, ...result.items];
    hasMoreData = result.items.length === count;
    currentPage += 1;
  }

  return allData;
};
```

Code 7: Fetch all pages function.

3.6.6 getUserFromSession.js

The getUserFromSession function retrieves the user information from the session storage of the browser, see code 8.

```
export const getUserFromSession = () => {
  const user = sessionStorage.getItem('user');
  return user? JSON.parse(user) : null;
};
```

Code 8: Get user from session function.

3.6.7 validation.js

This file contains validation functions to assist in validating user input. It includes the `isEmailValid` function, which uses a regular expression to verify the validity of a user's email address during account creation, see code 9.

```
export const isEmailValid = (email) => {  
  const emailRegex = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;  
  return emailRegex.test(email);  
};
```

Code 9: Email validation function.

4 Reflection

In this portfolio subproject, a modern, modular frontend application was designed and implemented using React, Bootstrap, and JavaScript, integrating seamlessly with the backend and database developed in earlier subprojects. A component-based architecture was adopted to ensure a clear separation of concerns throughout the project.

Key features include the use of React hooks to manage state efficiently, service functions for consistent API communication, and utility functions for reusability. Integration with The Movie Database (TMDB) API enriched the user experience by providing additional data, while toast notifications, dynamic routing, and intuitive navigation ensured usability in line with Nielsen's heuristics.

Despite initial challenges, such as aligning the backend API with frontend requirements and ensuring robust data validation, these issues were addressed through iterative development and collaboration. The emphasis on clean design and performance optimization resulted in a functional single-page application that fulfills the requirements of an interactive and user-friendly movie database system.

5 Appendix A

In the following section, each group member will reflect on the project and discuss how course concepts were applied to the product design or implementations.

5.1 Ida Hay Jørgensen

This is a reflection on the importance of performance in an application and how focusing on introducing a new library such as TanStack Query could enhance the application's performance. It will also discuss the existing setup with PostgreSQL, ASP.NET, and JavaScript data fetching and how this could be optimized further.

General performance considerations

Performance is a critical aspect of any application, directly influencing user experience, responsiveness, and overall satisfaction. A poorly performant application results in slow response times, higher bounce rates, and lowered user retention according to Mozilla (Developer Mozilla, 2024a).

Some of the key performance factors Mozilla lists include:

- Efficient database querying
- Optimized data fetching
- Reduced server response times
- Effective caching mechanisms

The existing setup of this project uses ASP.NET for backend development and PostgreSQL as the database, both of which are robust technologies well-suited for building high-performance applications. However, further optimization opportunities exist, as explored by Niket Shah (Shah, 2024) and Volkan Alkılıç (Alkılıç, 2023). Shah suggests that optimizations could include; enabling response compression and implementing caching, among other techniques. Alkılıç mentions that multiple parallel tasks can be set up to improve the querying of data, along with partitioning strategies to enhance performance.

Current optimizations and challenges

In the application, several strategies have been used to ensure backend performance such as asynchronous operations which use `async/await` to optimise backend processing and improve response times. Database indexing and query optimization are done in PostgreSQL to reduce latency and improve query performance. However, there is still room for further improvement.

On the frontend, data fetching is done with the native JavaScript `fetch` operation in combination with custom logic to handle the states in the application. While the `ApiClient` class abstracts the `fetch` operation and provides a structured way to handle API requests and responses, it still introduces some challenges which are outlined in the following:

- **Lack of caching:** Repeated network requests take a lot of resources and lead to redundant data transfers which lowers the overall performance.
- **Repetitive code:** Custom logic for data fetching and state handling results in a messy code base where redundant code appears.
- **Increased complexity:** Manual state managing, error handling, and background updates add complexity to the code base.

The application has performance limitations, as seen in the Lighthouse reports generated for both the media page and the person page, see figure 6 and figure 7. These reports show that the application scores in the medium to below-medium range on a scale of 0 to 100, with 100 being the most performant. This indicates significant room for optimization.

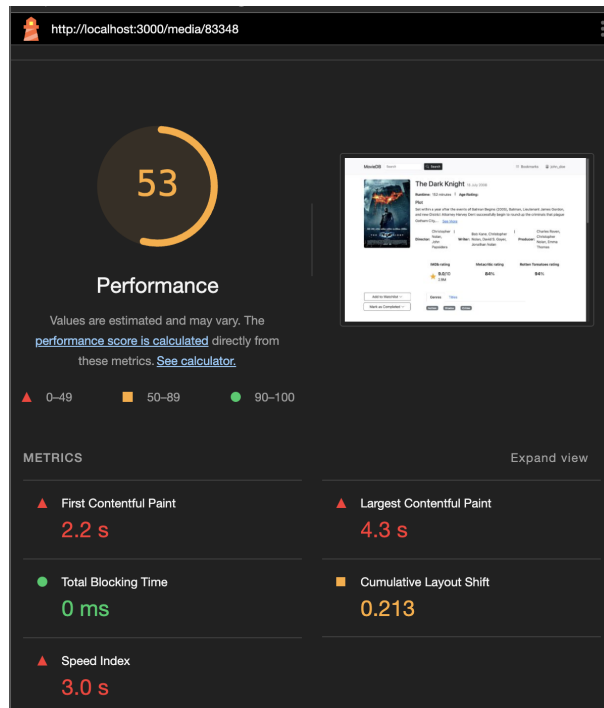


Figure 6: Lighthouse performance report, media page.

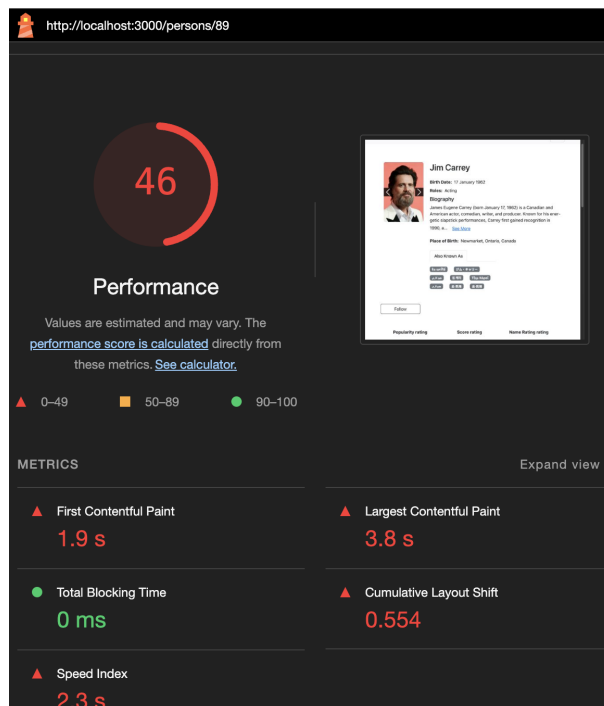


Figure 7: Lighthouse performance report, person page.

To address these challenges a data-fetching library like TanStack Query can be introduced. It simplifies asynchronous data fetching, state management, and caching which are critical for performance and could optimize the current application (TanStack, 2024).

This library has some benefits compared to manually implementing this logic yourself. Some of the benefits are automatically caching of data, to reduce repeated network requests. Simplified state management, to reduce code complexity. Background updates to data, which ensure fresh data across the application. Optimized pagination, to reduce the load on the server. These features align with the modern way of working and make for a cleaner and more robust code.

In code example 10 the usage of TanStack Query is shown, which is from TanStack's documentation. This shows the implementation of fetching data async and handling states while managing errors.


```
import {
  QueryClient,
  QueryClientProvider,
  useQuery,
} from '@tanstack/react-query';

const queryClient = new QueryClient();

export default function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <Example />
    </QueryClientProvider>
  );
}

function Example() {
  const { isPending, error, data } = useQuery({
    queryKey: ['repoData'],
    queryFn: () =>
      fetch('https://api.github.com/repos/TanStack/query').then((res) =>
        res.json(),
      ),
  });

  if (isPending) return 'Loading...';

  if (error) return 'An error has occurred: ' + error.message;

  return (
    <div>
      <h1>{data.name}</h1>
      <p>{data.description}</p>
      <strong>{data.subscribers_count}</strong>{' '}
      <strong>{data.stargazers_count}</strong>{' '}
      <strong>{data.forks_count}</strong>
    </div>
  );
}
```

Code 10: Fetching data using TanStack Query, TSX file, (TanStack, 2024).

First, an instance of the `QueryClient` is created. This instance manages all the queries, caching, and related functionality. The application is then wrapped with the `QueryClientProvider` so every component and page has access to the `QueryClient`.

Using the `useQuery` hook, the states `isPending`, `error`, and `data` can easily be handled. The `queryKey` and `queryFn` are also defined:

- The `queryKey` is a unique key that TanStack Query uses for caching and validating queries.
- The `queryFn` is the actual function where the endpoint is called, and the fetch operation is performed.

With this setup, conditional rendering in the UI can be performed. The states `isPending` and `error` ensure good user feedback while the data is being fetched or if an error occurs. Once the data is ready, it can be displayed in the UI.

This simple setup, as mentioned, can enhance the frontend code both in terms of performance and lowering complexity. This contributes to a better user experience by making the application more responsive and efficient.

5.2 Julius Krüger Madsen

Testing is a crucial part of software development, ensuring functionality, reliability, and performance. In our project, Testing was limited to the backend due to the way the course is structured. This primarily involved unit and integration tests, which helped validate core logic and ensure backend reliability. However, this limitation meant that frontend behavior and database interactions were not fully tested, leading to integration challenges later in the project. Throughout the process, we faced challenges and mistakes, such as ensuring consistent communication between the backend API and the frontend, which initially led to mismatched data formats and errors. These experiences show that we could have improved our testing processes a lot more.

For a larger-scale and long-term project, testing across various layers of the stack becomes even more critical. Comprehensive coverage ensures reliability at all levels, from UI testing for validating React components in the frontend, to unit and API testing for backend functionality, and integration testing for smooth communication between the frontend, backend, and database.

5.2.1 Database

The database for the project was built in PostgreSQL as the very first step of the project. While we did not implement any formal database tests, we relied on tools like the query analyzer to manually test and optimize procedures and functions. This allowed us to verify that queries returned the expected results and identify areas for optimization. Additionally, our script that executed all database functions and output their results, providing a very basic level of validation to confirm they worked as expected.

Although these approaches were helpful, more robust database testing techniques could have been beneficial:

- **ACID Compliance Testing:** Explicit tests for Atomicity, Consistency, Isolation, and Durability would demonstrate that the database reliably handles transactions, even under failure scenarios.

- **Stress and Load Testing:** While not as relevant for this project, testing the database under simulated high-load conditions would validate its scalability and performance, especially for real-world applications.

By implementing these techniques, we could have ensured a stronger, more reliable database foundation. ACID compliance testing, in particular, would have demonstrated transactional reliability and helped build confidence in the system's robustness.

The absence of comprehensive database tests led to challenges, such as the need to revisit and fix search functionality multiple times. Implementing the techniques mentioned above could have minimized these issues, reduced debugging time, and made schema modifications smoother and more predictable.

5.2.2 Backend

In second part of the project, the backend, we implemented three primary tests, focusing on specific functionalities:

- **JWT Token Generation:** We tested the `JwtTokenGenerator` class to ensure that JWT tokens were correctly generated, parsed, and contained the expected values.
- **Login Management:** The `LoginManager` was tested to ensure proper handling of login attempts, including verifying passwords and returning JWT tokens when authentication was successful.
- **Person Manager:** Tests validated repository interactions and ensured that data retrieval methods worked correctly. For instance, mapping results from the database was confirmed for queries like `GetPersonsAsync`.

During this project in an effort to improve our process, we implemented a test needed label for tasks. This label ensured that tests were written and verified before any task could be closed. This approach aligned well with our agile and iterative development approach we took, where continuous testing and feedback are essential for delivering a stable product.

5.2.3 Frontend

In my previous work during my student job and as part of my bachelor's degree, I gained experience with UI testing using Selenium. This experience showed me the importance of automated UI tests. While we did not implement UI testing in this project due to time constraints and the scale of the frontend, incorporating tools like Selenium in future projects would significantly improve development efficiency. Automated tests would help validate UI components, catch regressions early, and allow us to make changes with greater confidence by quickly identifying issues. In this project, all component changes had to be manually tested, which was time-consuming and less reliable. By introducing automated UI tests, we could ensure that components behave as expected.

Overall, while we implemented the most essential tests in the backend, we could have gained a lot from doing, automated testing, stress testing, and ACID compliance checks, which would have helped us.

5.3 Marek Laslo

I would like to provide feedback on the overall project planning and implementation with my team. From the beginning, we decided to adopt Agile development methodologies. Since we are a small team, we implemented only certain aspects of Scrum and used a KANBAN board to monitor tasks throughout the project. We also defined constraints for working with Git, established naming conventions, and set up an overall workflow. Based on retrospective sessions held after each section hand-in, we adjusted the workflow as needed. This process worked very well for me and made our team function like a well-oiled machine, for the most part.

Adopting Agile principles and working in iterations seemed like a good and modern approach to project development. Having prior experience with Agile, this approach felt very natural to me. However, I also experienced moments of realization where the actual separation of the project into three main parts resembled a traditional waterfall approach. Essentially, we first built the database, then the API, and finally the frontend. We had very little foresight regarding how the requirements for subsequent parts might change. While this approach worked well initially, I noticed that when we reached the frontend development phase, many bugs and functionality issues in the backend needed fixing. This was sometimes frustrating because I wanted to focus on learning and working on the frontend but had to switch back to resolving backend issues. Although this was still a valuable learning experience, I wonder if the workflow could have been smoother with a more integrated Agile approach.

Moving on to individual topics, I would like to discuss SQL procedures and functions. Honestly, as it had been three years since I completed my bachelor's degree—and I hadn't done much coding or learning in the interim—this concept was quite new to me. Initially, I thought, "Wow, this looks cool—I can create functions directly in the database!" However, by the end of the project, it seemed to me that this might not be the best way to approach things in our context. While I can imagine scenarios where using stored procedures and functions would make sense, in the context of this portfolio, it felt counterproductive to have business logic split between the database and the API.

I recall searching through the API implementation to locate the search functionality, only to realize that it was implemented as stored functions in the database. Personally,

I would prefer to keep all business logic within the API because, as a project grows, it becomes easier to lose track of what is implemented where. Consolidating business logic within the API would make it easier to maintain and scale the application over time.

Next, I would like to address my experience with Entity Framework. I don't remember having any previous experience with ORMs, but if I did, it was likely during my AP classes quite some time ago. When I first heard about Entity Framework, I thought to myself, "Why would I use it if I can write raw queries?" Looking back, I think I was more naive at the beginning of the course than I am now.

Entity Framework seems like a great tool for building APIs. It helps protect against SQL injection and makes it easy to map relationships within a database. Once the mapping is done, working with data manipulation and transfer becomes very straightforward. I found the workflow incredibly enjoyable once I understood the mechanics of building an API using Entity Framework and .NET.

Currently, I am assisting a startup company in building an internal system that includes both the database and API while simultaneously learning new concepts in this course. For their API, they chose the Express.js framework. I helped them implement it using Express.js while comparing the process to ASP.NET and Entity Framework. I quickly realized how much easier the process is with ASP.NET and Entity Framework, and I kind of regret not knowing about it earlier in the development process.

Lastly, I would like to share my experience working with JavaScript. I really enjoy the language, but I enjoy TypeScript even more. I feel that TypeScript makes frontend development more secure by introducing static typing, which reduces the need for excessive runtime validation.

Beyond that, I thoroughly enjoyed learning new concepts in React, particularly the use of hooks, which were somewhat new to me. My previous experiences with React had not been very positive, but this project significantly improved my perspective. I feel that I learned a great deal—not just from my classes but also from the professional experiences and insights shared by my teammates.

5.4 Sofus Hilfling Nielsen

The core project for the course involved recreating a movie database application, inspired by platforms like IMDb and TMDb. This required designing and implementing a three-tier architecture encompassing a PostgreSQL database, a .NET 8 backend, and a React-based frontend.

Three-tiers The three-tier architecture is commonly used when developing applications like the movie database. While it doesn't scale horizontally as effectively as microservice or message-based architectures, it compensates for this by being much simpler to maintain and develop features for while still allowing for complex integration with external systems. The separation into three distinct tiers helps to maintain a clear boundary between different parts of the system and allows for creating a complex multi-functional system Ingeno, 2018.

While developing the frontend we quickly realized that the results from our media search weren't relevant compared to what we expected. This required us to rewrite some of the search functions in the database. This highlighted the benefits of a more agile approach, where the database design evolves alongside the backend and frontend as the scope increases or changes. It should be noted however, that this poses other difficulties since changing a SQL schema later in development can cause compatibility problems between new versions of the schema and existing versions of the backend.

To enhance security and user experience, we implemented an authentication system using short-lived access tokens (15 minutes) and longer-lived refresh tokens (12 hours). While refresh tokens were managed in backend memory for practicality, a production-grade system should store token data in persistent storage to avoid token loss during server restarts.

As we developed the frontend, we realized the API lacked certain features needed for a smooth user experience. For example, we extended the endpoint for retrieving user bookmarks to include media information, which reduced the number of API calls required to fetch related data. However, this resulted in mixed resource types being returned from some endpoints. GraphQL could be an alternative to a RESTful design worth exploring in similar projects. It allows clients to specify the data they need in

each request, avoiding over-fetching and reducing backend load while possibly enhancing performance and reducing load times.

Another challenge arose with a project requirement to fetch person images directly from TMDb via the frontend. Since React operates on the client side, exposing the API key in the frontend introduces a security risk. A better solution would be to handle this integration in the backend, allowing it to securely retrieve image urls and pass only the necessary data to the frontend.

Containers To maintain consistency during development, we containerized the backend and database using Docker. This ensured that all team members were working on the same versions, even as updates and bug fixes to the backend and database were applied. This approach proved very useful as the frontend development required multiple changes to the backend or database throughout development of the frontend.

While containers helped us to ensure consistency across our different environments it also introduced its own challenges, such as port conflict on local machines on initial setup, which required manual configurations of each environment before the containers could run. We also had to manually build and push the container images to a shared container registry every time changes were made to the codebase. A Continuous Integration and Continuous Deployment (CI/CD) pipeline could have helped alleviate this problem Red Hat, 2023.

Besides the benefits we saw during development the biggest advantages from introducing containers is to streamline the deployment process for a complex IT system like the movie database. Containers help throughout deployment by being scalable (easy to spin up new container instances), portable, and makes it easy to control which version is deployed in different environments through image tags Sharma, 2023.

Conclusion The collaborative nature of the project emphasized the importance of teamwork in developing complex IT systems. The scope of the project presented real-world challenges such as optimizing database queries, addressing API performance issues, and ensuring environment consistency during development. These challenges emphasized the value of modern practices like CI/CD pipelines and containerization for ensuring scalability and efficiency.

6 Appendix B

6.1 Wireframes

MovieDB 🔍 [Sing up](#) [Log in](#)

Sign up

Email

Username

Password

Figure 8: Low fi sign up page design, (Own-work, 2024).

MovieDB

Sing up Log in

Welcome to MovieDB

Search...

Search

Figure 9: Low fi home page design, (Own-work, 2024).

MovieDB

Sing up Log in

Title

year

Movie length

age rating

Plot

It is a long established fact that a reader will be distracted by the readable content of a page when looking at its layout. The point of using Lorem Ipsum is that it has a more-or-less normal distribution of letters, as opposed to using 'Content here, content here', making it look like readable English. **See more**

director

composer

screen writer

watchlist

↓

add a note

Completed

Rotten Tomato

1/4

Meta critics

1/4

User Score

1/4

Genres

Alternativ titles

Crew

Cast

me

Name

Name

Name

Name

Name

Similar movies

me

Name

Name

Name

Name

Name

Extra information

Type

Awards

Episodes

Figure 10: Low fi media id page design, (Own-work, 2024).

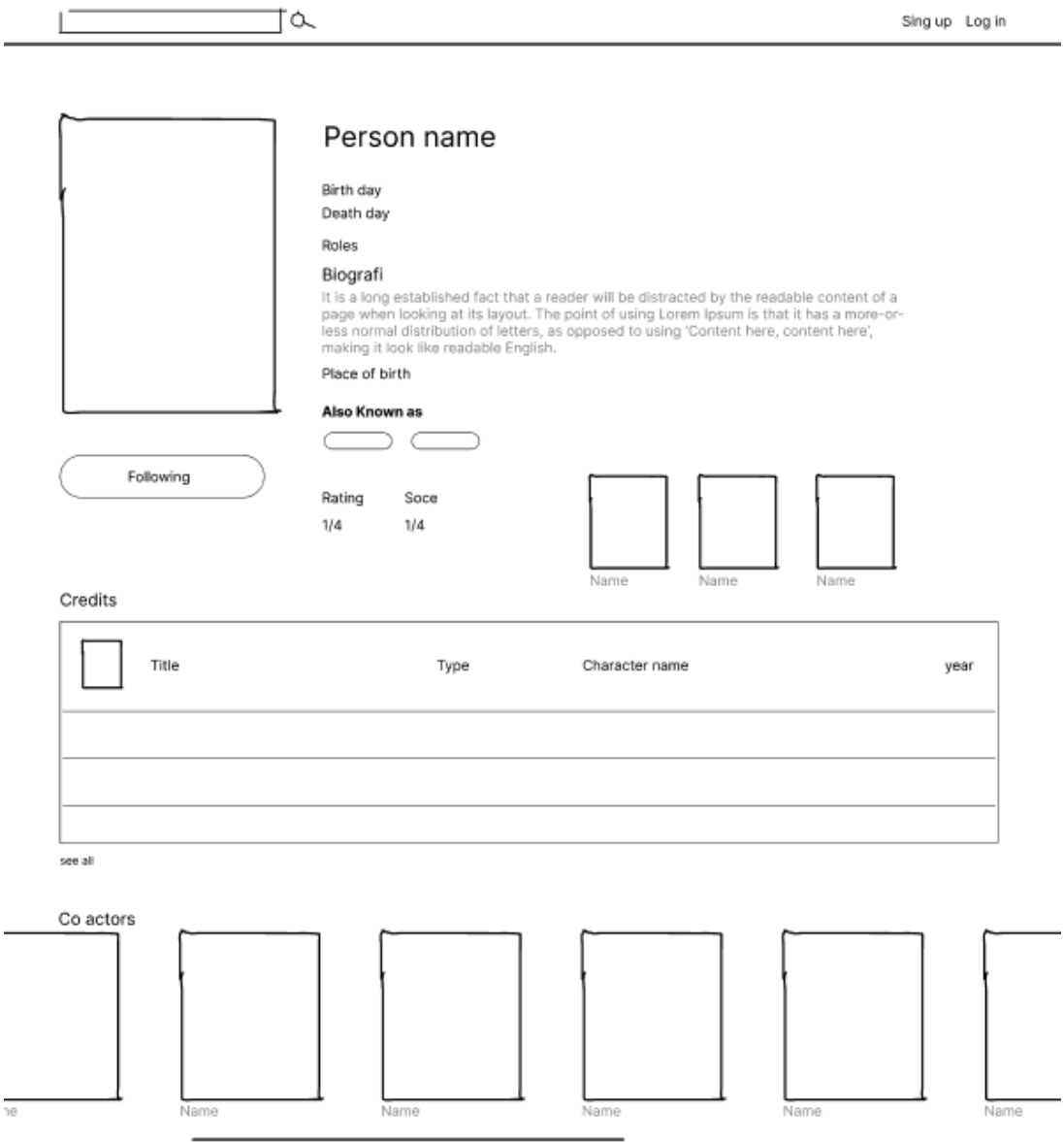


Figure 11: Low fi persons id page design, (Own-work, 2024).

MovieDB

Sing up Log in

Search

Search Type

Simple

tom

Search

Titles

Name

Name

Name

Name

Name

See More

Prev

23456

Next

Persons

Name

Name

Name

Name

Name

See More

Prev

23456

Next

Figure 12: Low fi search page design, (Own-work, 2024).

MovieDB

user

Welcome {username }

Following

Bookmarked

Completed

	Person name	date

Figure 13: Low fi following list page design, (Own-work, 2024).

MovieDB

user

Welcome {username }

Following **Bookmarked** Completed

	Title	Type	Date	Age Rating	Runtime	Note

Figure 14: Low fi bookmarked list page design, (Own-work, 2024).

MovieDB

Welcome {username }

FollowingBookmarkedCompleted

	Title	Type	Date	Re-watchability	Score	Review Text

Figure 15: Low fi completed list page design, (Own-work, 2024).

MovieDB 🔍

:≡ ○ user

Profile settings

Username

Email

Password

Sucess: you changed your data

Figure 16: Low fi profile page design, (Own-work, 2024).

6.2 ApiClient code

```
export class ApiClient {
  #baseUrl;
  #request;

  constructor(baseUrl, accessKey, options) {
    const url = baseUrl ?? import.meta.env.VITE_API_BASE_URL;
    if (url === undefined)
      throw new Error('Environment variable VITE_API_BASE_URL missing');
    this.#baseUrl = new URL(url);

    this.#request = {
      headers: new Headers({
        'Content-Type': 'application/json',
      }),
    };
  }
}
```

```
};

const token = accessKey ?? getCookie('token');
if (token !== undefined) {
  this.#request.headers.set('Authorization', `Bearer ${token}`);
}

if (options !== undefined) {
  if (options.headers !== undefined) {
    for (const [key, value] of Object.entries(options.headers)) {
      this.#request.headers.set(key, value);
    }
  }
}

}

// params format: [{key: 'key1', value: 'value1'}, {key: 'key2',
↪ value: 'value2'}]
#_getUrl(path, params) {
  const url = new URL(path, this.#baseUrl);
  if (params !== undefined)
    for (const param of params) {
      url.searchParams.append(param.key, param.value);
    }
  return url;
}

async Get(path, params) {
  const requestUrl = this.#_getUrl(path, params);
  const response = await fetch(requestUrl, this.#request);
  const returnResp = new ApiResponse(response.status);
  if (response.ok) {
    try {
```

```
        returnResp.value = await response.json();
      } catch {} // eslint-disable-line no-empty
    }
    return returnResp;
  }

  async Post(path, data, params) {
    const req = {
      ...this.#request,
      method: 'POST',
    };

    if (data !== undefined) {
      req.body = JSON.stringify(data);
    }

    const requestUrl = this.#getUrl(path, params);
    const response = await fetch(requestUrl, req);
    const returnResp = new ApiResponse(response.status);
    if (response.ok) {
      try {
        returnResp.value = await response.json();
      } catch {} // eslint-disable-line no-empty
    }
    return returnResp;
  }

  async Patch(path, data) {
    const req = {
      ...this.#request,
      body: JSON.stringify(data),
      method: 'PATCH',
    };
  }
```

```
const requestUrl = this._getUrl(path);
const response = await fetch(requestUrl, req);
const returnResp = new ApiResponse(response.status);
if (response.ok) {
  try {
    returnResp.value = await response.json();
  } catch {} // eslint-disable-line no-empty
}
return returnResp;
}

async Delete(path) {
  const req = {
    ...this.#request,
    method: 'DELETE',
  };
  const requestUrl = this._getUrl(path);
  const response = await fetch(requestUrl, req);
  const returnResp = new ApiResponse(response.status);
  if (response.ok) {
    try {
      const text = await response.text();
      if (text) {
        returnResp.value = JSON.parse(text);
      }
    } catch (error) {
      console.error('Error parsing response:', error);
    }
  }
  return returnResp;
}
```

6.3 Changes to the database and the backend

6.3.1 Portfolio subproject 1 - Database

This section details changes, including new feature and bug fixes, made to the database to support the frontend. Git tags has been used to distinguish between the new changes and the original submission, see. [GitHub Releases - portfolio_subproject_1](#)

Features:

- Add: Extend user table with salt column to support password hashing.
- Add: Containerize database with Dockerfile. Including script for automatically applying database schema on container startup.
- Add: Update search history as side-effects of all search queries (add `userId` param to functions missing it)
- Add: review text to dummy data for entities in completed table.
- Add: following entries for `john_doe` dummy user
- Add: Allow search query without `userId`, by skipping insert in search history table if null.
- Add: Script for dropping all functions from database.
- Add: Additional indexes for media and person tables (increase search performance)

Bug fixes:

- Fix: Release data on release table does not allow null
- Fix: Primary information for media not set if no release or poster exist in original dataset (use left join)
- Fix: During data transformation release data is converted to 0001-01-01 BC which is not readable by CSharp.

- Fix: DO block for inserting seasons has invalid syntax
- Fix: `structured_string_search` query not working if some params are null.
- Fix: Order results for search queries by the number of votes for the media (return more relevant results)
- Fix: character column in `cast_member` table not normalized

6.3.2 Portfolio subproject 2 - Backend

This section details changes, including new features, enhancements, and bug fixes, made to the backend/api to support specific requirements in the frontend. Git tags has been used to distinguish between the new changes and the original submission, see. [GitHub Releases - portfolio_subproject_2](#)

Features and enhancements:

- Add: hashing of password when signing up and when signing in. (500000 hasing iterations using SHA256)
- Add: Allow CORS for `'http://localhost:3000'` and `'https://localhost:3000'`
- Add: Containerize api with Dockerfile. Build and push image to GitHub.
- Add: Include basic media/person result for bookmark, completed and follow lists to decresase number of required api calls from frontend to backend.
- Add: Include user score on entities in completed list result
- Add: Pagination for all endpoints that return lists of items (genre, language, country & titles)
- Add: Filter for name on person endpoint
- Add: Include `SeasonId` and `SeriesId` on `Episode` and `Season` entities respectively.
- Refactor: Generalize pagination into generic `PagingResult<T>` type

- Refactor: HATEOAS link generation

Bug fixes:

- Fix: order of paginated results vary between calls leading to skipped entries.
- Fix: Search functions to match changes made to database.
- Fix: persons/{id}/media endpoint to include cast and crew information
- Fix: Some entities missing HATEOAS links
- Fix: Episode database model referring to Media instead of Season

References

- Alkılıç, V. (2023). Maximizing postgresql performance: Advanced features and techniques [Accessed: 17/12/2024]. <https://dev.to/volkanalkilic/maximizing-postgresql-performance-advanced-features-and-techniques-50pe>
- Anderson, T. (2023). How to structure a website: What it is, why it matters, and best practices [Accessed: 16/12/2024]. <https://www.bluehost.com/blog/website-structuring/>
- Developer Mozilla. (2024a). The "why" of web performance [Accessed: 17/12/2024]. https://developer.mozilla.org/en-US/docs/Learn/Performance/why_web_performance
- Developer Mozilla. (2024b). Spa (single-page application) [Accessed: 17/12/2024]. <https://developer.mozilla.org/en-US/docs/Glossary/SPA>
- Ingeno, J. (2018, August). *Software architect's handbook*. Packt Publishing.
- Nielsen, J. (1994). 10 usability heuristics for user interface design [Accessed: 05/12/2024]. <https://www.nngroup.com/articles/ten-usability-heuristics/>
- Own-work. (2024). Application wireframing [Accessed: 16/12/2024]. <https://www.figma.com/design/R5qjDjDpJqpTQ18iX2LhF8/Wireframe-portfolio-3?node-id=1-474&t=Mpo9eX1TroYZ9vyR-1>
- Red Hat. (2023, December). <https://www.redhat.com/en/topics/devops/what-is-ci-cd>
- Shah, N. (2024). Performance optimization techniques for asp.net core applications [Accessed: 17/12/2024]. <https://www.techcronus.com/blog/performance-optimization-techniques-for-asp-net-core-applications/>
- Sharma, R. (2023, November). <https://container-registry.com/posts/container-image-versioning/>
- TanStack. (2024). Tanstack query [Accessed: 17/12/2024]. <https://tanstack.com/query/latest/docs/framework/react/overview>